

Soutenance · 10 minutes

Implémentation d'un modèle de Régression Logistique

Projet Mathématiques–Informatique · Python · Jupyter Notebook

Université Aix-Marseille

Enseignante : Raquel Urena

Équipe :

Katia Alili : @artshleyy

Alex Ayivi : @AYIVIAlex

Hawa Konta : @lolipopiz

Kenza Benmansour : @kenzalittlegenius



Idée centrale

Transformer une combinaison linéaire en probabilité, puis apprendre les meilleurs paramètres.

Attentes du projet

Implémentation depuis zéro

Pas de bibliothèque de Machine Learning : les fonctions essentielles sont codées manuellement.

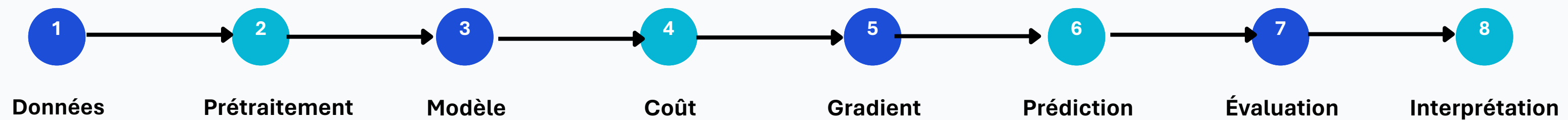
Entraînement

Apprentissage des paramètres w et b grâce à la descente de gradient.

Évaluation

Mesures : matrice de confusion, accuracy, précision, rappel et F1-score.

Fil conducteur du projet :



Problème traité :

Le modèle répond à une question binaire :

Classe 0 ou classe 1 ?

y = 0

y = 1

Dans le notebook, la cible y est encodée en 0/1, et les variables explicatives sont regroupées dans X.

- **X** : ensemble des variables explicatives utilisées pour prédire la pluie.
- **Y** : variable cible (*Rain*), représentant la présence ou l'absence de pluie.

```
#Séparation des variables  
  
#variable explicative  
X = data[["Temperature", "Humidity", "Cloud_Cover"]]  
  
#variable cible  
Y = data["Rain"]
```

```
#Encodage de La cible "Rain" : on encode la variable en nombres  
data["Rain"] = data["Rain"].map({"no rain": 0, "rain": 1})  
  
#Matrice de corrélation  
sns.heatmap(data.corr(), annot=True, cmap="coolwarm")  
plt.show()
```

X : variables explicatives

Matrice de taille $m \times n$: m observations et n variables.

y : cible binaire

Vecteur de taille m contenant seulement 0 ou 1.

Chargement et exploration des données

Chargement des données météo

```
#Importation du data set
dataset = pd.read_csv("weather_forecast_data.csv")
```

```
# Création d'une copie du dataset original.
# Toutes les modifications seront effectuées sur cette copie.
data = dataset.copy()
```

Exploration des données

```
: #Les cinq premiere lignes du dataset
data.head()
```

	Temperature	Humidity	Wind_Speed	Cloud_Cover	Pressure	Rain
0	23.720338	89.592641	7.335604	50.501694	1032.378759	rain
1	27.879734	46.489704	5.952484	4.990053	992.614190	no rain
2	25.069084	83.072843	1.371992	14.855784	1007.231620	no rain
3	23.622080	74.367758	7.050551	67.255282	982.632013	rain
4	20.591370	96.858822	4.643921	47.676444	980.825142	no rain

```
: print("Répartition de la variable cible :\n")

proportions = data["Rain"].value_counts(normalize=True) * 100

print(f"No rain : {proportions['no rain']:.2f}%")
print(f"Rain    : {proportions['rain']:.2f}%")
```

Répartition de la variable cible :

No rain : 87.44%
Rain : 12.56%

Valeurs aberrantes

```
# Vérification de la présence de valeurs aberrantes (méthode IQR)
```

```
for col in data.select_dtypes(include="number").columns:
    Q1 = data[col].quantile(0.25)
    Q3 = data[col].quantile(0.75)
    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    outliers = ((data[col] < lower_bound) | (data[col] > upper_bound))

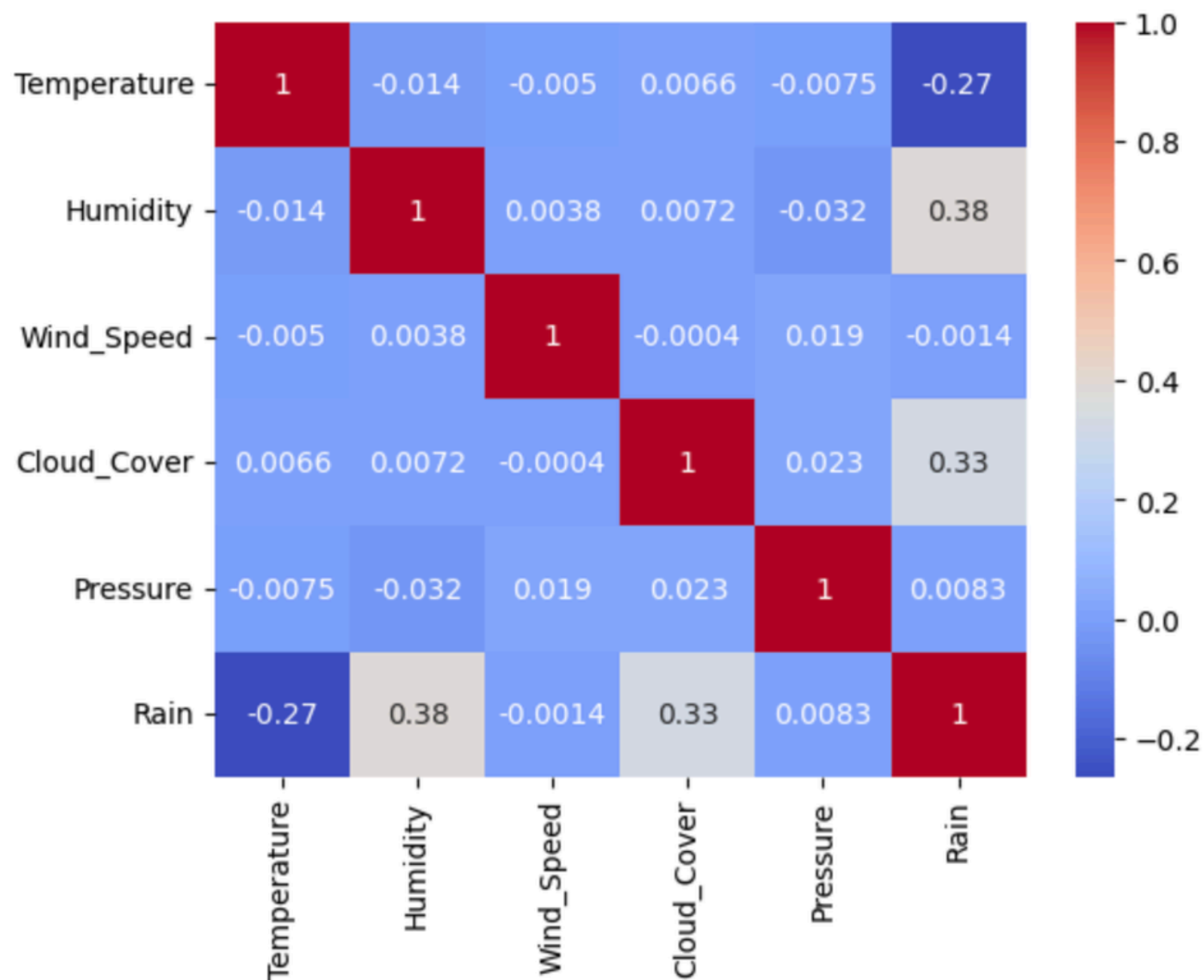
    print(f"{col}: {outliers} outliers")
```

Prétraitement des données

Rendre les données compatibles avec la descente de gradient

```
#Encodage de la cible "Rain" : on encode la variable en nombres
data["Rain"] = data["Rain"].map({"no rain": 0, "rain": 1})

#Matrice de corrélation
sns.heatmap(data.corr(), annot=True, cmap="coolwarm")
plt.show()
```



1

Encodage

Transformer la cible Rain en 0 ou 1 et les variables catégorielles en valeurs numériques.

2

Matrice de corrélation

Détermination des variables caractéristiques à garder (features)

3

Découpage

Séparer les données en train, validation et test.

4

Standardisation

Standardiser les variables sans utiliser les données de validation et de test.

```
mean = X_train.mean()
std = X_train.std()
X_train = (X_train - mean) / std
X_val = (X_val - mean) / std
X_test = (X_test - mean) / std
```

mean et std sont calculés uniquement sur X_train, puis réutilisés tels quels pour X_val et X_test, jamais recalculés sur ces ensembles.

Entraînement, validation et test

Pourquoi ne pas évaluer sur les mêmes données que celles utilisées pour apprendre



Train

Données utilisées pour apprendre w et b .

Validation

Données utilisées pour suivre le comportement du modèle et choisir des hyperparamètres.

Test

Données gardées pour l'évaluation finale, une fois les choix terminés.

Le test final doit représenter des données que le modèle n'a jamais vues pendant l'apprentissage.

Modèle mathématique

Une combinaison linéaire transformée en probabilité

Pour chaque observation x , on calcule d'abord un score linéaire :

$$z = w^T x + b$$

Puis ce score est transformé par la fonction sigmoïde :

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Décision finale : si $\hat{y} \geq t$, on prédit la classe 1 ; sinon, la classe 0. Ici, t représente le seuil de décision.

Interprétation

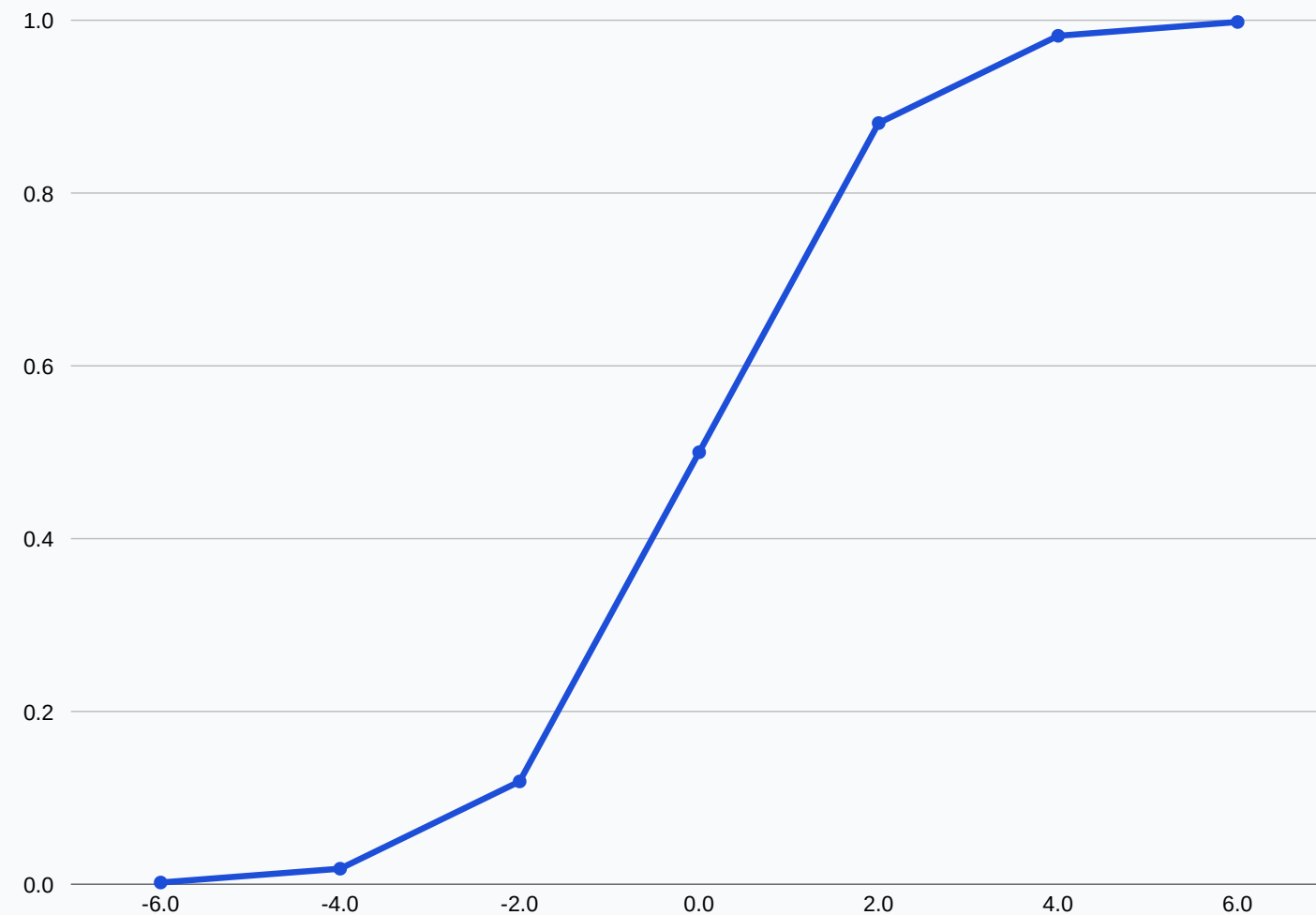
- w contient un poids par variable.
- b est le biais : il décale la frontière de décision.
- $\hat{y}$ est une probabilité entre 0 et 1.

```
# Prédiction classe : convertit les probabilités en classes (0 ou 1).
def predict(X, w, b, threshold):
    # Comparaison des probabilités au seuil de décision.
    return (predict_proba(X, w, b) >= threshold).astype(int)
```


Fonction sigmoïde

Le passage du score brut à une probabilité

```
: # Fonction Sigmoïde : calcule de la probabilité prédite à partir de z
def sigmoid(z):
    return 1 / (1 + np.exp(-z))
```



Lecture de la courbe

- Si z est très négatif, $\sigma(z)$ est proche de 0.
- Si $z = 0$, la probabilité vaut 0,5.
- Si z est très positif, $\sigma(z)$ est proche de 1.

Elle permet donc de relier un modèle linéaire à une probabilité interprétable.

Fonction de coût : Log-Loss

Mesurer l'erreur sur des probabilités

$$J(w, b) = -\frac{1}{m} \sum_{i=1}^m [y_i \log(a_i) + (1 - y_i) \log(1 - a_i)]$$

a_i est la probabilité prédite pour l'observation i .

Bonne prédiction

Si $y = 1$ et a est proche de 1, la perte est faible.

Mauvaise prédiction

Si $y = 1$ et a est proche de 0, la perte devient très grande.

Précaution numérique

On utilise un petit ε ou `np.clip` pour éviter $\log(0)$, qui n'est pas défini.

C'est une solution simple à un problème fréquent lors de l'implémentation manuelle.

```
: # Fonction coût (log-loss)
def compute_cost(X, y, w, b):
    # Nombre d'exemples d'entraînement.
    m = len(y)

    # Calcul de la combinaison linéaire.
    z = X @ w + b
    # Calcul des probabilités prédites.
    a = sigmoid(z)

    # Évite log(0), qui provoquerait une erreur numérique.
    epsilon = 1e-15
    a = np.clip(a, epsilon, 1 - epsilon)

    # Calcul de la fonction coût (log-loss).
    cost = -(1/m) * np.sum(y * np.log(a) + (1 - y) * np.log(1 - a))
    return cost
```

Objectif de l'apprentissage : trouver w et b qui minimisent `compute_cost(X, y, w, b)`.

Structure de l'implémentation

Fonctions codées manuellement dans le notebook

sigmoid(z)

Convertit un score en probabilité.

compute_gradients(X, y, a)

Calcule dw et db.

compute_loss(y, a)

Calcule la log-loss moyenne.

evaluate(y, y_pred)

Calcule la matrice de confusion et les scores.

train_logistic_regression(...)

Effectue la descente de gradient, sauvegarde les coûts et retourne les paramètres.

predict(X, w, b)

Transforme les probabilités en classes.

```
# Fonction de calcul des gradients
def compute_gradients(X, y, w, b):
    # Nombre d'exemples d'entraînement.
    m = len(y)

    # Calcul de la combinaison linéaire.
    z = X @ w + b
    # Calcul des probabilités prédites.
    a = sigmoid(z)

    # Gradient par rapport aux poids
    dw = (1/m) * X.T @ (a - y)
    # Gradient par rapport au biais
    db = (1/m) * np.sum(a - y)

    return dw, db
```

Cette fonction traduit directement les formules mathématiques vues précédemment :

$z = X @ w + b \rightarrow$ le score linéaire

$a = \text{sigmoid}(z) \rightarrow$ la probabilité prédite

dw, db $\rightarrow$ les dérivées de la fonction coût par rapport à w et b

Le résultat (dw, db) est ensuite utilisé pour mettre à jour les paramètres à chaque itération de la descente de gradient

Descente de gradient

Comment les paramètres apprennent à chaque itération

1

Prédire

$$a = \sigma(Xw + b)$$

2

Comparer

$$\text{erreur} = a - y$$

3

Dériver

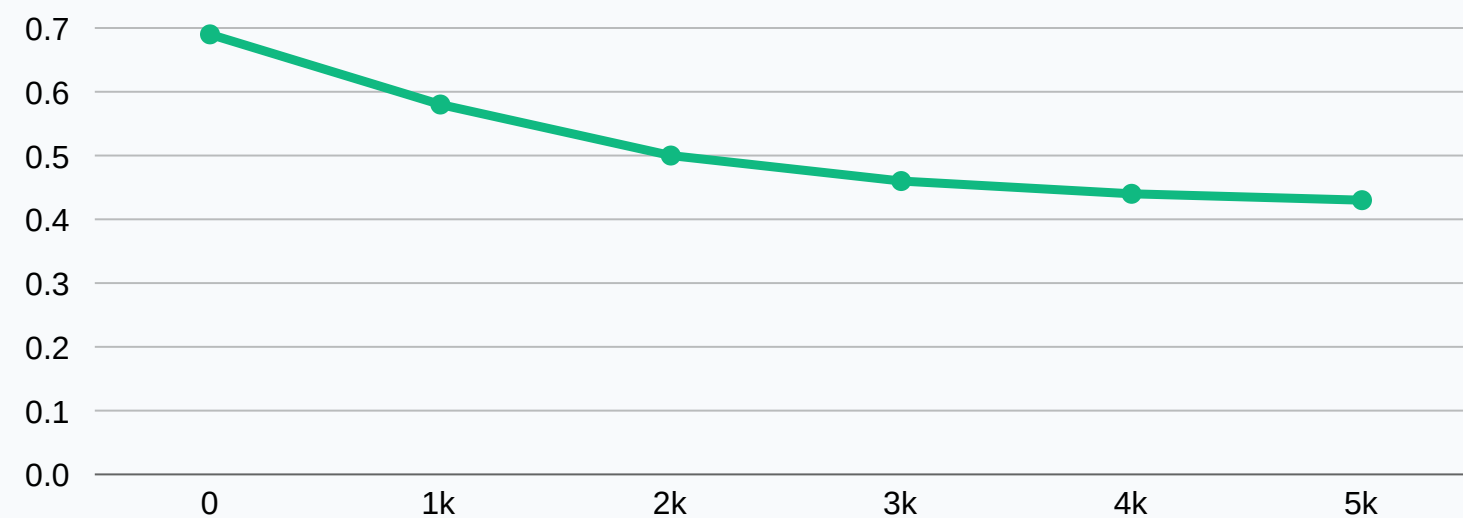
$$dw = 1/m X^T(a-y), db = 1/m \sum(a-y)$$

4

Mettre à jour

$$w \leftarrow w - \alpha dw, \\ b \leftarrow b - \alpha db$$

α = learning rate



C'est le pas d'apprentissage.
S'il est trop grand, le coût peut diverger ;
s'il est trop petit, l'apprentissage devient très lent.

Forme attendue : la fonction coût diminue puis se stabilise.

Early stopping

L'entraînement s'arrête automatiquement quand le coût de validation cesse de s'améliorer. Cela évite le surapprentissage et réduit le temps de calcul, tout en conservant les meilleurs paramètres observés.

```
min_delta = 1e-5

if best_val_cost - val_cost > min_delta:
    best_val_cost = val_cost
    best_w = w.copy()
    best_b = b
    patience_counter = 0
else:
    patience_counter += 1

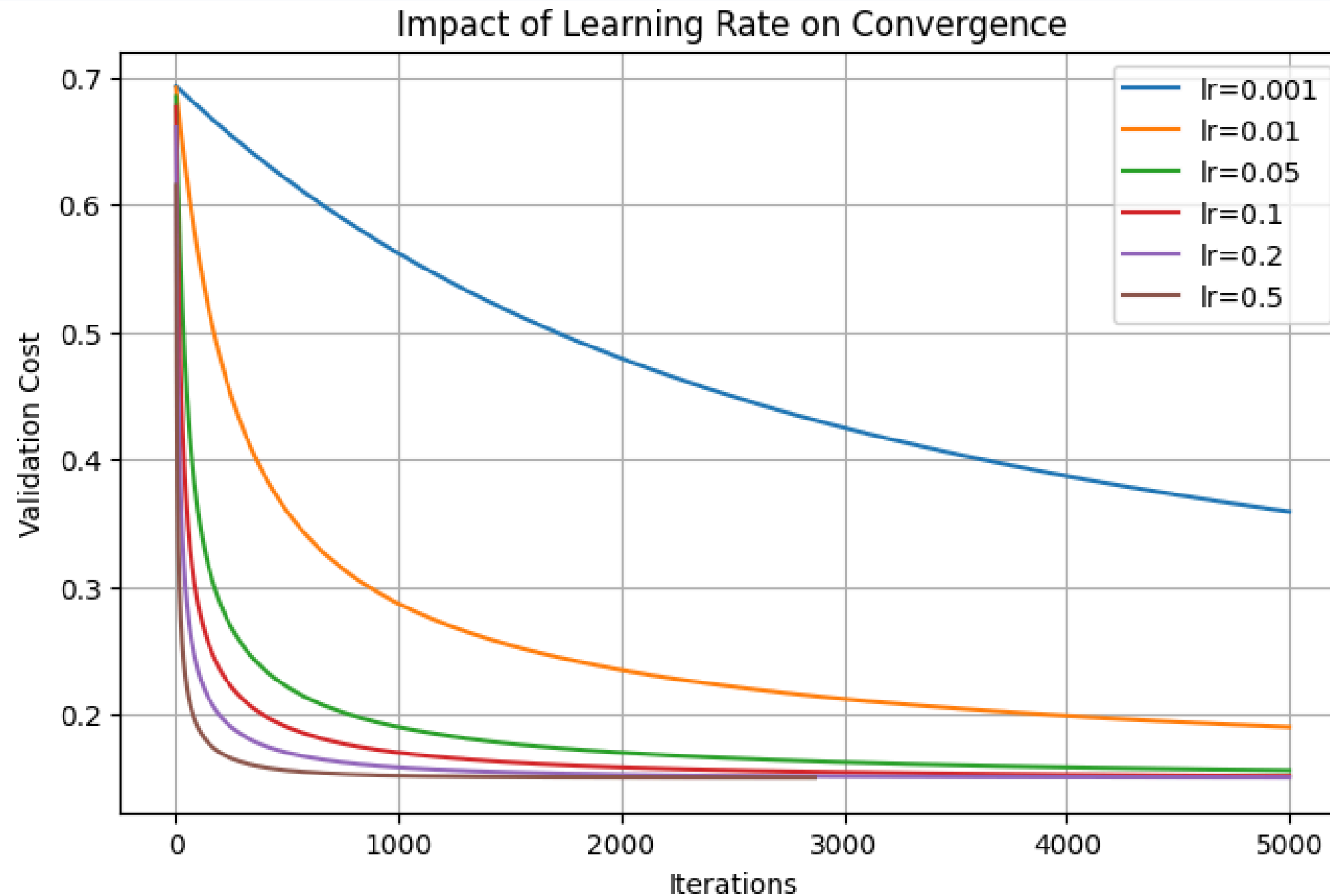
if patience_counter >= patience:
    print(
        f"Early stopping à l'itération {i} "
        f"pour learning_rate={learning_rate}" )

    break

return best_w, best_b, train_costs, val_costs
```

Impact du learning rate

Un learning rate trop faible (0.001) ralentit fortement la convergence.
Au-delà de 0.05, les courbes convergent presque toutes vers la même valeur.
Le gain devient marginal.



```
learning_rates = [0.001, 0.01, 0.05, 0.1, 0.2, 0.5]

plt.figure(figsize=(8,5))

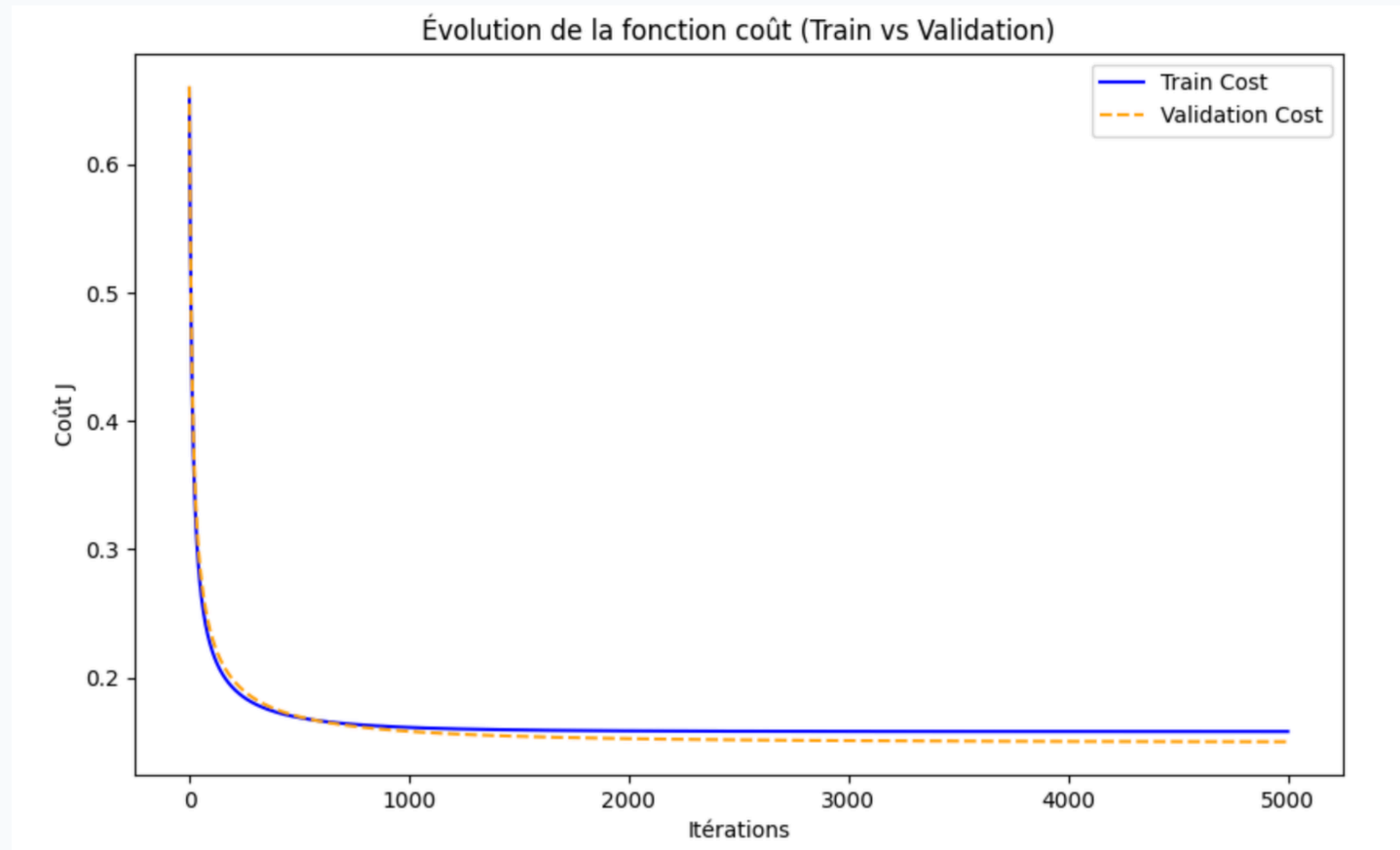
for lr in learning_rates:

    w, b, train_costs, val_costs = train_logistic_regression(
        X_train,
        Y_train,
        X_val,
        Y_val,
        learning_rate=lr,
        n_iterations=5000
    )

    plt.plot(val_costs, label=f"lr={lr}")

plt.xlabel("Iterations")
plt.ylabel("Validation Cost")
plt.title("Impact of Learning Rate on Convergence")
plt.legend()
plt.grid(True)
plt.show()
```


Visualisation de l'apprentissage du modèle



Lecture du graphique

- Axe horizontal : nombre d'itérations
- Axe vertical : valeur du coût
- Bleu : entraînement
- Orange : validation

Interprétation

Les deux courbes sont presque superposées.

Le modèle généralise bien et présente peu de surapprentissage.

Évaluation du modèle

Passer des prédictions aux indicateurs de performance

Matrice de confusion

```
Matrice de confusion :  
[[318  15]  
 [  7  35]]
```

```
Accuracy   : 0.9413  
Précision  : 0.7000  
Rappel     : 0.8333  
F1-score   : 0.7609
```

Indicateurs

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

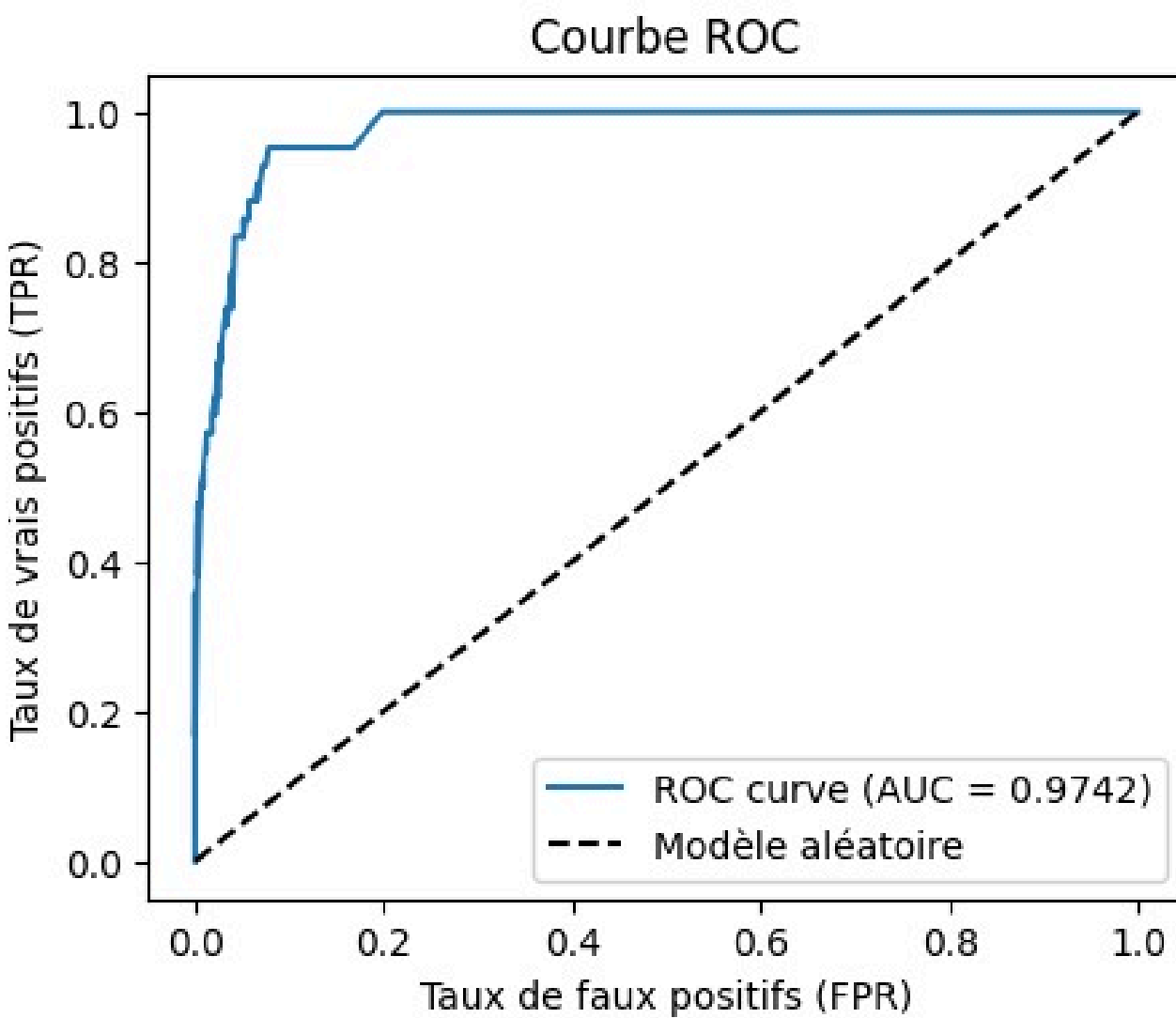
$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$F1 = 2 \frac{Precision \times Recall}{Precision + Recall}$$

Courbe ROC et AUC

Évaluer le modèle indépendamment du seuil choisi



AUC : 0.9742

Pourquoi cette courbe ?

Le dataset est déséquilibré (~87% no rain / 13% rain). Dans ce contexte, l'accuracy seule est trompeuse : un modèle qui prédirait toujours "no rain" obtiendrait déjà 87% sans rien apprendre.

Ce qu'elle mesure

La courbe ROC teste tous les seuils possibles et trace le taux de vrais positifs contre le taux de faux positifs. Plus elle est proche du coin supérieur gauche, meilleur est le modèle.

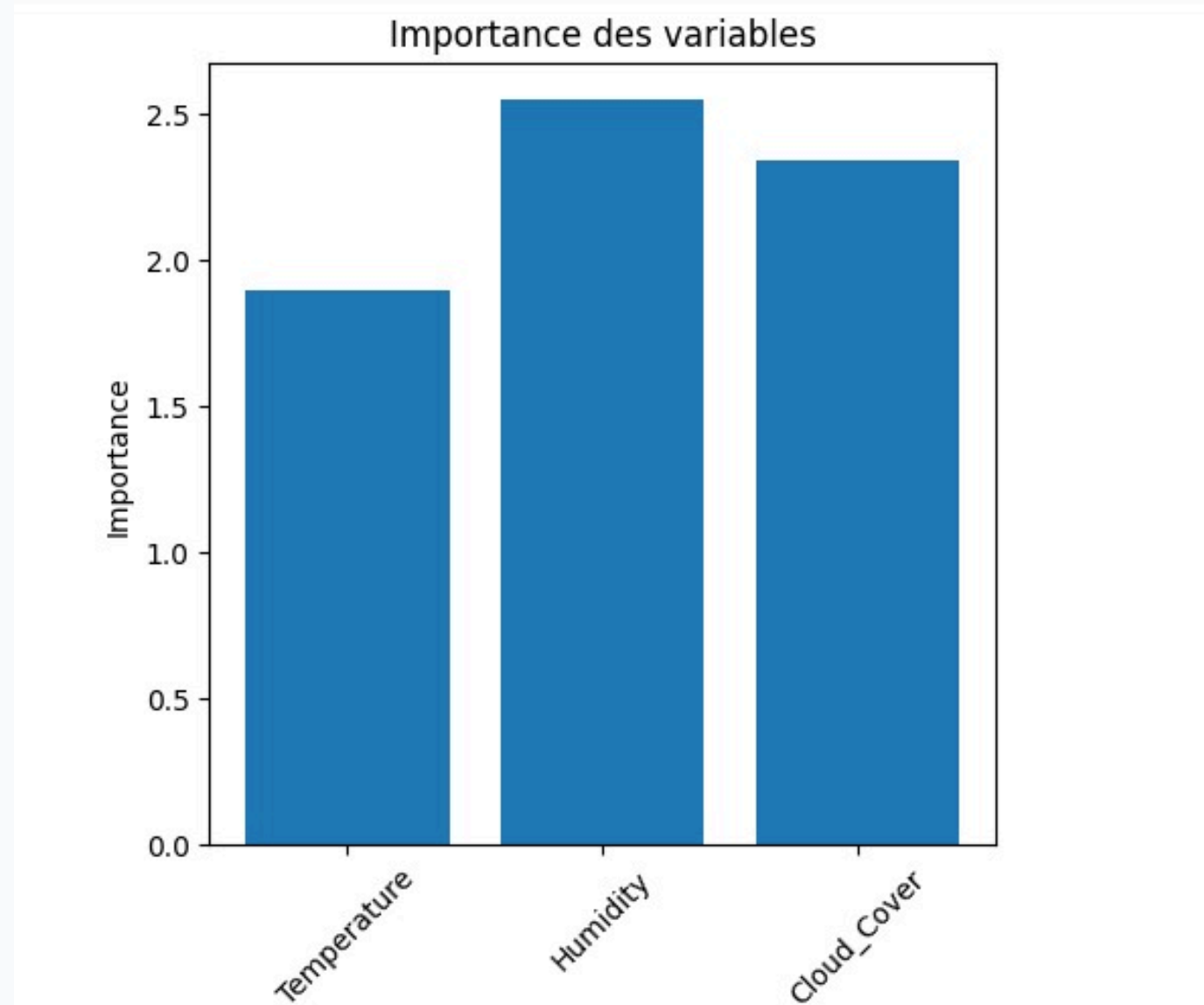
AUC=0.9742

Un modèle aléatoire aurait une AUC de 0.5. Ici, le modèle discrimine très bien les deux classes.

Ce résultat nuance le F1-score modéré observé précédemment (69.39%), qui s'explique surtout par le déséquilibre des classes, pas par une faiblesse du modèle.

Interprétation des coefficients

Comprendre quelles variables influencent la prédiction



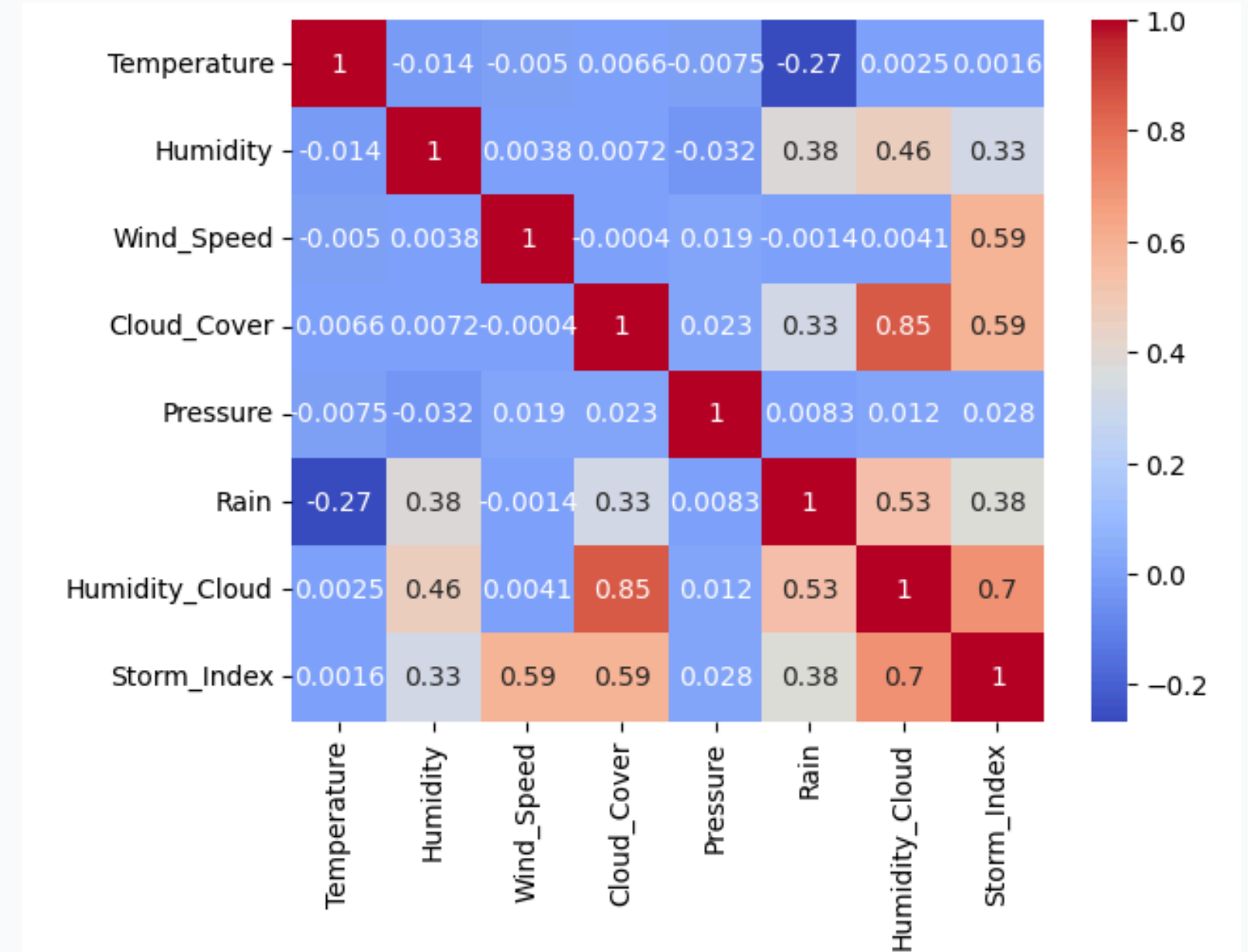
Comment lire w ?

- $w > 0$: augmente la probabilité de la classe 1.
- $w < 0$: diminue la probabilité de la classe 1.
- $|w|$ grand : variable plus influente.

Après standardisation, comparer les coefficients est plus pertinent car les variables sont sur une échelle comparable.

Amélioration possible du modèle

```
# --- FEATURE ENGINEERING ---  
#Créons de nouvelles colonnes pour améliorer notre modèle  
#La pluie est souvent associée à la présence simultanée d'humidité et de nuages.  
data["Humidity_Cloud"] = data["Humidity"] * data["Cloud_Cover"] / 100  
  
#Indicateur de temps orageux  
data["Storm_Index"] = (  
    data["Humidity"]  
    * data["Wind_Speed"]  
    * data["Cloud_Cover"]  
)
```



Pourquoi ces variables ?

Humidity_Cloud combine humidité et nébulosité, deux facteurs souvent associés à la pluie.

Storm_Index capture un effet conjoint humidité × vent × nuages, qui peut signaler un système orageux.

Ces nouvelles variables sont plus fortement corrélées à Rain (0.53 et 0.38) que les variables individuelles, ce qui suggère qu'elles peuvent améliorer la capacité prédictive du modèle.

Défis rencontrés et solutions apportées

Montrer la démarche de résolution, pas seulement le résultat

Défi

Implémenter sans scikit-learn

Calculer les gradients

Éviter les erreurs numériques

Choisir les hyperparamètres

Interpréter les résultats

Solution

Découper le modèle en petites fonctions vérifiables.

Repartir des formules et vérifier les dimensions de X , y , w .

Utiliser ε / `np.clip` dans la log-loss.

Comparer plusieurs learning rates et observer la courbe du coût.

Afficher la matrice de confusion et les coefficients.

Ces difficultés sont utiles pédagogiquement : elles montrent que nous avons compris le fonctionnement interne du modèle.

Améliorations observées

On remarque une amélioration de l'AUC

```
AUC : 0.9765
```

On remarque également une amélioration de la matrice de confusion et des indicateurs de performance

```
Matrice de confusion :  
[[320  13]  
 [  7  35]]
```

```
Accuracy   : 0.9467  
Précision  : 0.7292  
Rappel     : 0.8333  
F1-score   : 0.7778
```


Limites et conclusion

Analyse critique du modèle et bilan du projet

Limite 1 : linéarité

La régression logistique sépare les classes avec une frontière linéaire dans l'espace des variables.

Limite 2 : qualité des données

Le modèle dépend fortement du choix des variables, de l'encodage et de la normalisation.

Limite 3 : classes déséquilibrées

Si une classe est rare, l'accuracy peut donner une impression trop optimiste.

Bilan

- Nous avons implémenté toutes les étapes principales du modèle.
- Nous avons entraîné les paramètres par descente de gradient.
- Nous avons évalué les performances et interprété les coefficients.

Merci pour votre attention